

Basic Structure of C# Program

In this article, I am going to discuss the **Basic Structure of the C# Program** using a Console Application. Please read our previous article before proceeding to this article where we discussed the [Introduction & Environment Setup for C# Application](#) development. As part of this article, I am going to discuss the following pointers in detail.

1. **What is C#.NET?**
2. **Advantages of using the .NET Framework from the C# point of view.**
3. **Different Types of applications are developed using C#.NET.**
4. **What is the visual studio?**
5. **What is a console application?**
6. **How to Create a console application using visual studio?**
7. **Understanding the Basic Structure of a C# Program.**
 - Importing section**
 - Namespace Declaration**
 - Class Declaration**
 - Main() method**

So, here, first, we will understand what is C#.NET and Visual Studio and what type of applications we can develop using C#.Net. Then we will discuss the basic structure of a C# program using a console application.

What is C#.NET?

1. C#.NET is one of the Microsoft Programming Languages to work with the .NET Framework, .NET Core, or .NET to develop different kinds of applications such as Web, Console, Windows, Restful Services, etc.
2. It is the most powerful programming language among all programming languages available in the .NET framework because it contains all the features of C++, VB.NET, and JAVA, and also has some additional features. As we progress in this course, you will come to know the additional features.
3. C#.NET is a completely Object-Oriented Programming Language. It means it supports all 4 OOPs Principles i.e. Abstraction, Encapsulation, Inheritance, and Polymorphism.

Based on the features, we can define **C# as just a Simple, Secure, Robust, Portable, Platform-Independent, Architectural Neutral, Multithreaded, Automatic Memory Management, Object-Oriented Programming Language with a strong type Exception Handling mechanism for developing different kinds of applications such as Web, Windows Form, Console, Web Services, Mobile Apps, etc. which can be run on different Operating Systems such as Windows, Linus, and Mac.**

Different Types of Applications are Developed using C#.NET.

1. **Windows Applications**
2. **Web Applications**
3. **Restful Web Services**
4. **SOAP Based Services**
5. **Console Applications**
6. **Class Library, ETC.**

What is the Visual Studio?

Visual Studio is one of the Microsoft IDE tools. Using this tool, we can develop, build, compile, publish and run applications with the .NET framework. This tool provides some features such as

1. **Editor**
2. **Compiler**
3. **Interpreters, and Many More**

What is a Console Application?

1. A console application is an application that can be run in the command prompt. For any beginner on .NET or anyone who wants to learn C# Language or anyone who wants to become an expert in C# Language, building a console application is ideally the first step to learning the C# Language.
2. The Console Applications contain a similar user interface to the Operating systems like MS-DOS, UNIX, etc.
3. The Console Application is known as the CUI application because in this application we completely work with the CUI environment.
4. These applications are similar to C or C++ applications.
5. Console applications do not provide any GUI facilities like the Mouse Pointer, Colors, Buttons, Menu Bars, etc.

Basic Structure of the C# Program

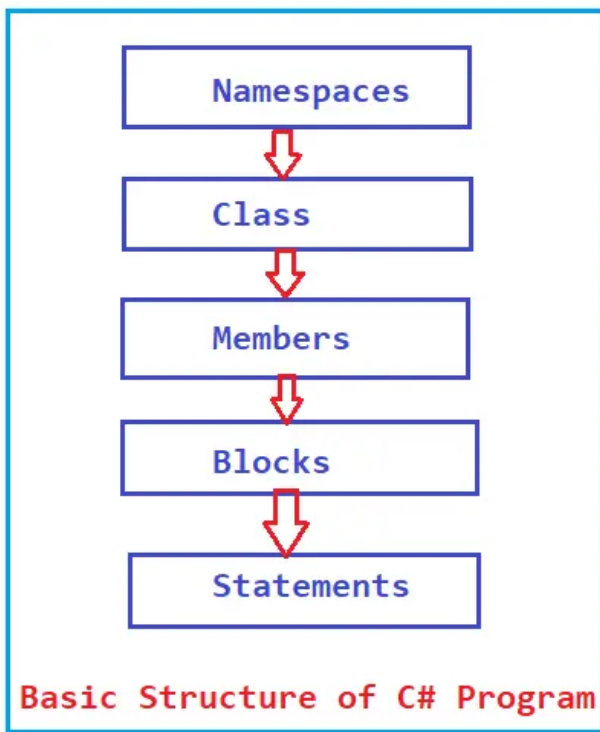
Now, let's understand the Basic Structure of the C# Program using a Console Application.

```
//Include classlibraries/namespaces
using System;

//Declare namespace
namespace namespaceName
{
    //Class Declaration
    class ClassName
    {
        //Data Members
        int SomeMember1;
        float SomeMember2;

        //Functions/Methods/Blocks
        static void SomeBlock()
        {
            //Statements
            Console.WriteLine("Hello World");
        }
    }
}
```

The above process is shown in the below diagram.



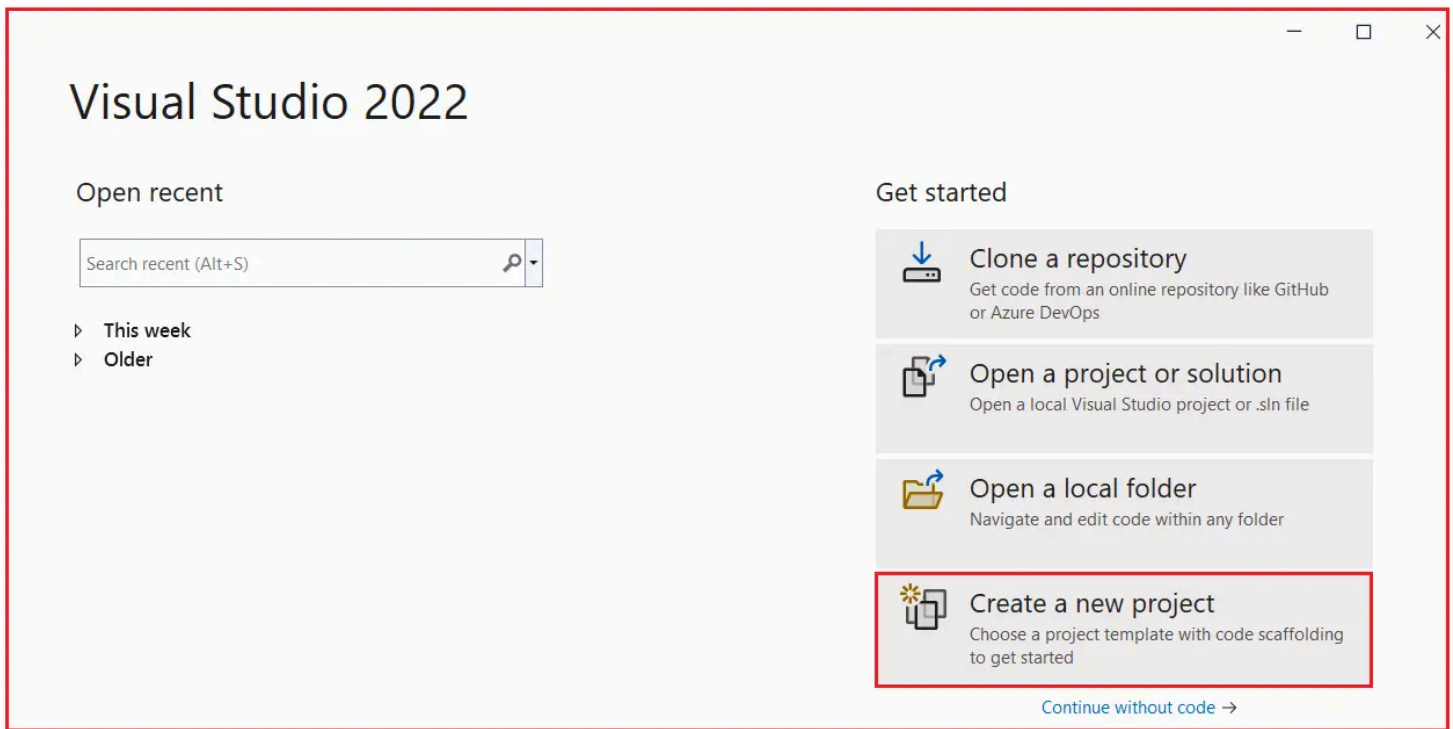
Note: C#.NET is a Case-Sensitive Language and Every Statement in C# should end with a Semicolon.

Example to Understand the Basic Structure of a C# Program:

Now, we are going to use Visual Studio to create a Console-Type Project. Then we are going to use the console application to display the message “**Welcome to C#.NET**”. Then, we will also see how to build and run the Console Application using Visual Studio GUI.

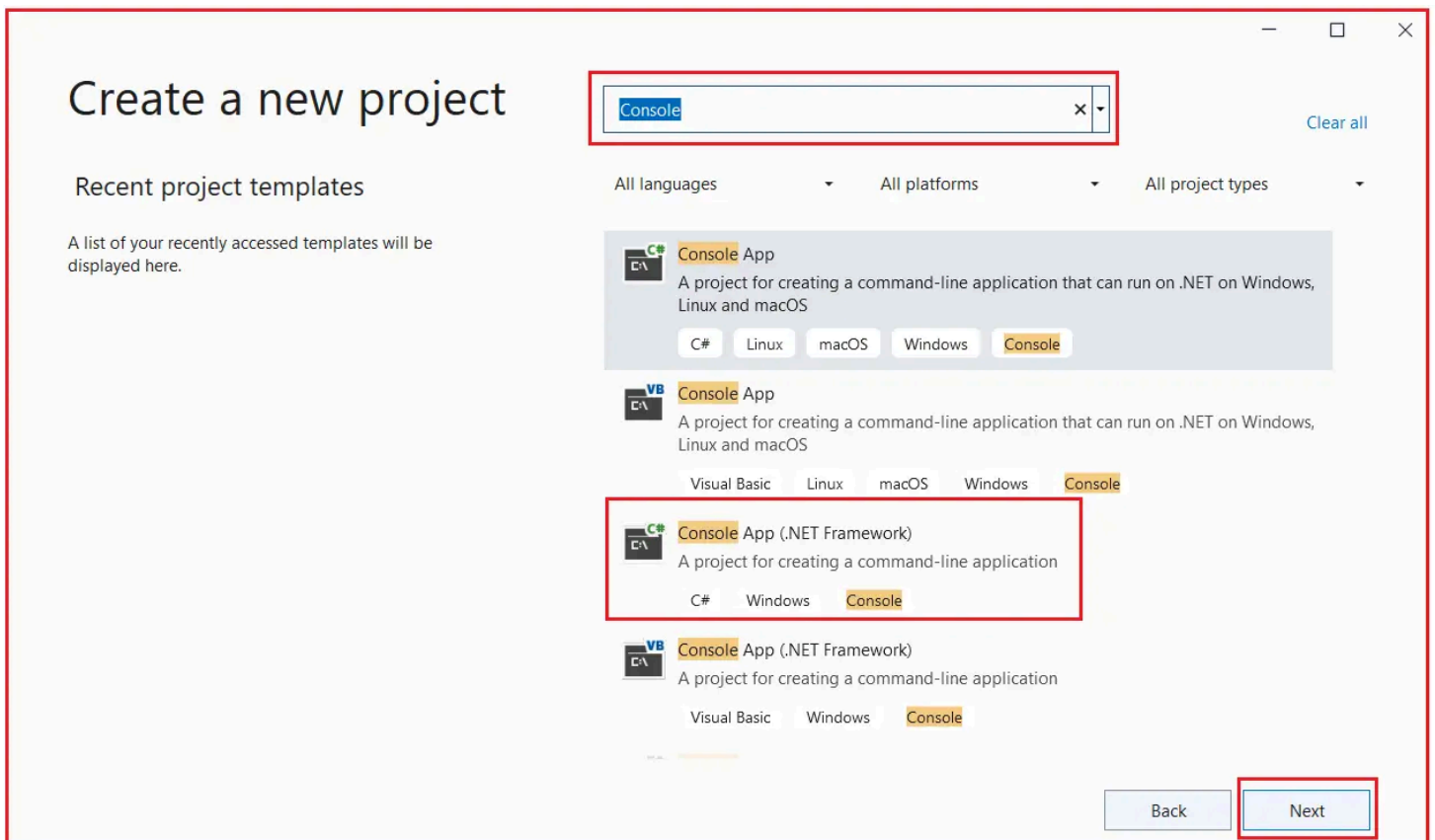
Step1

First, open Visual Studio 2022 (the latest version at this point in time) and then click on the **Create a New Project** option as shown in the below image.



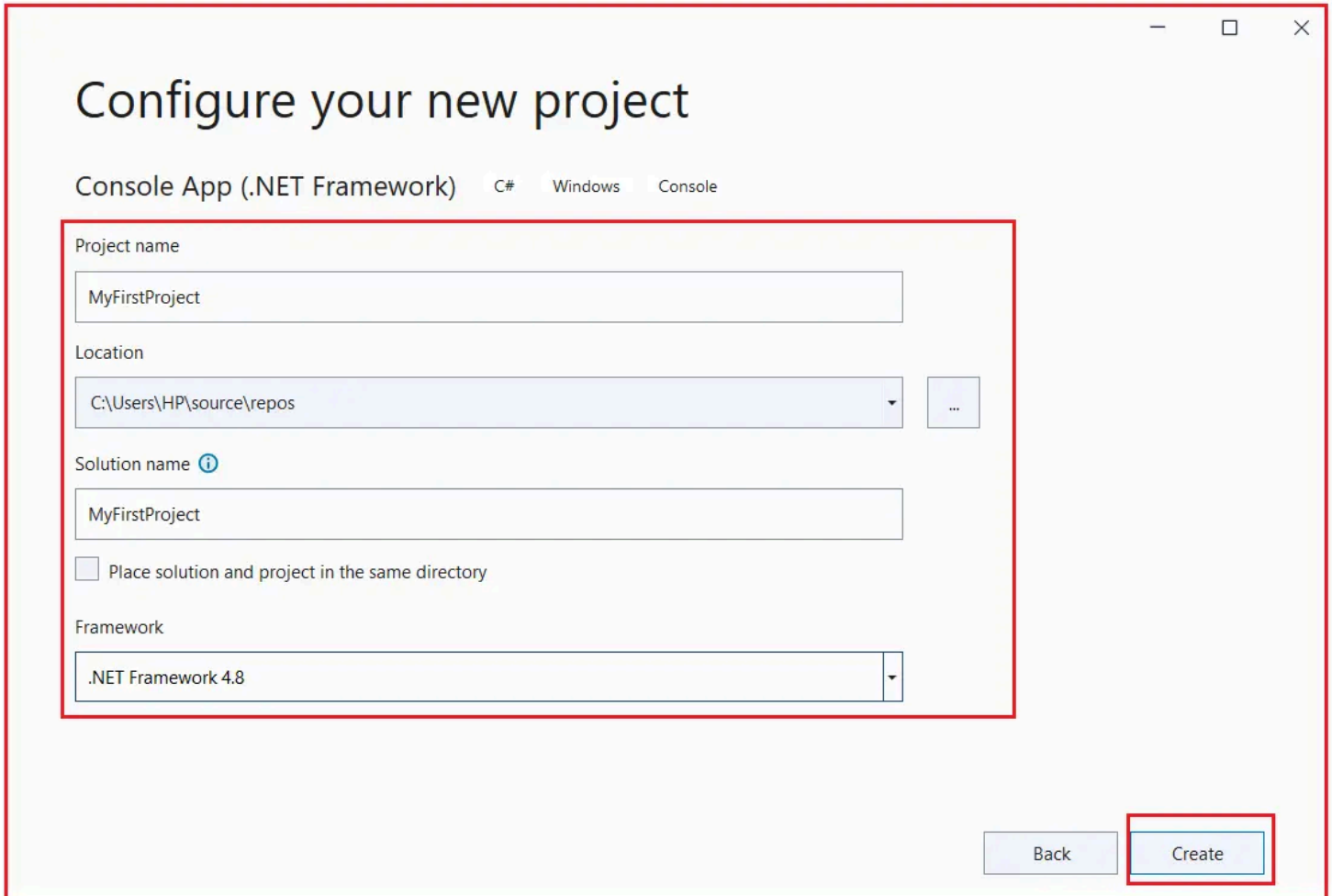
Step2

The next step is to choose the project type as a **Console Application**. Type Console in the search bar and you will see different types of console applications using C#, VB, and F# Programming Languages and using both .NET Framework and .NET Core / .NET. Here, I am selecting **Console App (.NET Framework)** using **C# Language** and then clicking on the **Next** button as shown in the below image.



Step3

The next step is to configure the new project. Here, you need to provide the project name and solution name. You can also give the same name to both project and solution but it is not mandatory. Here, I am providing the name **MyFirstProject** to both project and solution. You need to provide the location where you need to create the project. Here, you also need to provide the .NET Framework version that you want to use in this application. The latest version of the .NET Framework is 4.8. So, I am selecting **.NET Framework 4.8** and then clicking on the Create button as shown in the below image.



Configure your new project

Console App (.NET Framework) C# Windows Console

Project name

MyFirstProject

Location

C:\Users\HP\source\repos

Solution name ⓘ

MyFirstProject

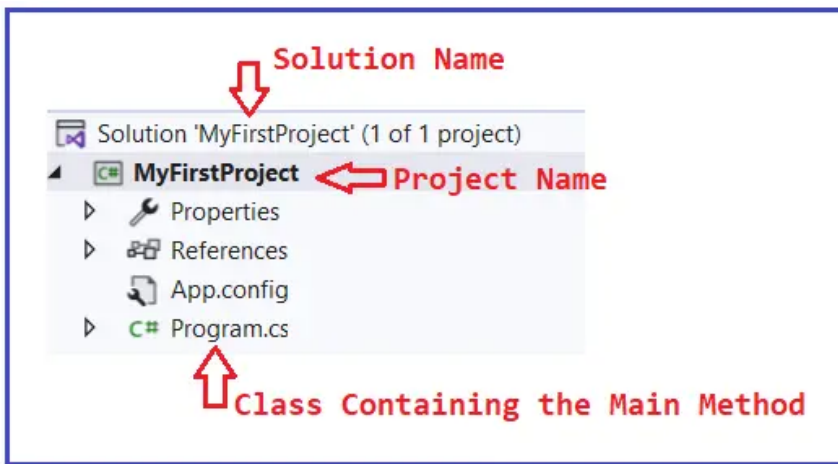
☐ Place solution and project in the same directory

Framework

.NET Framework 4.8

Back Create

Once you click on the Create button, visual studio will create the Console Application with the following structure.



A project called **MYFirstProject** will be created in Visual Studio. This project will contain all the necessary required files (Properties, References, App.Config files) to run the Console application. The Main program called **Program.cs** is the default code file that is created when a new console application is created in Visual Studio it contains the Main method by default and from that Main method our application is going to start its execution, but if you want you can also change this default behavior. So, if you look at the **Program.cs** class file, then you will see the following code.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
        }
    }
}
```

Step4

Now let's write our code which will be used to display the message "**Welcome to C#.NET**" in the console window. To do so, modify the Main method of the Program class as shown in the below code.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace MyFirstProject
{
    internal class Program
    {
```

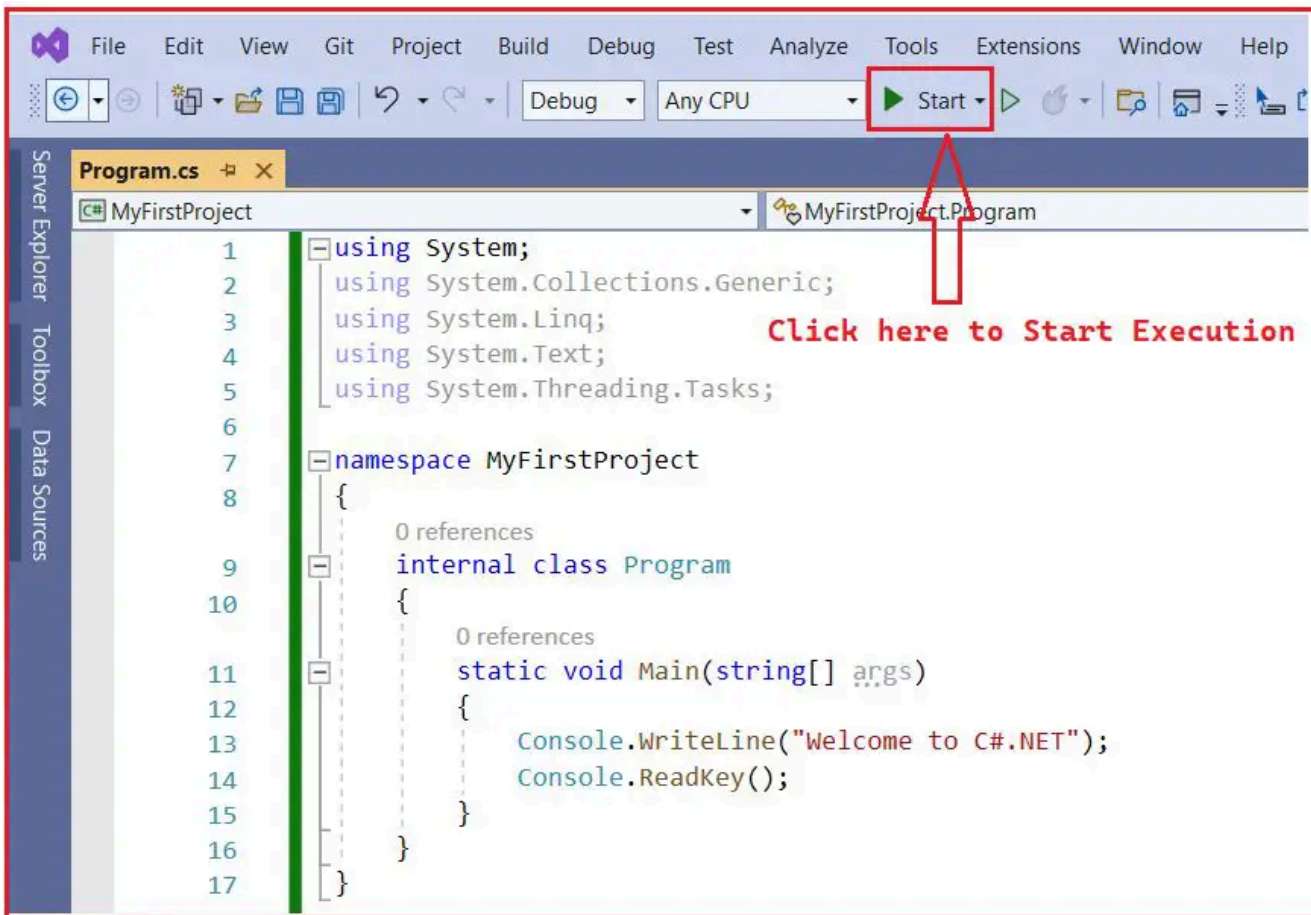
```

static void Main(string[] args)
{
    Console.WriteLine("Welcome to C#.NET");
    Console.ReadKey();
}
}
}

```

Step5

The next step is to run the .NET Application. To run any program in Visual Studio, you just need to click on the Start button or you can press **CTRL+F5** as shown in the below image.



Once you click on the Start button, you should get the following console window showing the message.



Understanding the Code:

Using Visual Studio, if we are creating a console application (excluding .NET 6), then automatically we are getting four sections which are shown in the below image.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

← Importing Namespaces Section

```
namespace MyFirstProject
{
```

← Namespace Declaration Section

```
    internal class Program
    {
```

← Class Declaration Section

```
        static void Main(string[] args)
        {
            Console.WriteLine("Welcome to C#.NET");
            Console.ReadKey();
        }
```

← Main Method Section

```
    }
```

```
}
```

Let's understand each of these sections in detail.

Importing Namespace Section:

This section contains importing statements that are used to import the BCL (Base Class Libraries) as well as user-defined namespaces if required. This is similar to the included statements in the C programming language. Suppose you want to use some classes and interfaces in your code, then you have to include the namespace(s) from where these classes and interfaces are defined. For example, if you are going to use the Console class in your code, then you have to include the System namespace as the Console class belongs to the System namespace.

Syntax: `using NamespaceName;`

Example: `using System;`

If the required namespace is a member of another namespace, we have to specify the parent and child namespaces separated by a dot as follows:

`using System.Data;`

`using System.IO;`

Note: A namespace is a container that contains a group of related classes and interfaces, as well as, a namespace can also contain other namespaces.

Namespace Declaration Section:

Here a user-defined namespace is declared. In .NET applications, all classes and interfaces or any type related to the project should be declared inside some namespace. Generally, we put all the related classes under one namespace and in a project, we can create multiple namespaces.

Syntax: `namespace NamespaceName {}`

Example: `namespace MyFirstProject {}`

Generally, the namespace name will be the same as the project name but it is not mandatory, you can give any user-defined name to the namespace.

Class Declaration Section:

For every Desktop Application in .NET, we need a start-up class file. For example, for every .NET Desktop Application like Console and Windows, there should be a Start-Up class that should have the Main method from where the program execution is going to start. When we create a Console Application using Visual Studio, by default, Visual Studio will Create the Start-Up class file with the name Program.cs which will have a class with the name Program that contains the Main method. A start-up class is nothing but a class that contains a Main() method from which the program execution is going to start.

Syntax:

```
class ClassName
{
}
```

Example:

```
class Program
{
}
```

Note: You can change this default behavior. You can create your own class and you can also make that class as the Start-Up Class by including the Main method. In our upcoming articles, we will discuss this in detail.

Main() Method Section:

The main() method is the entry point or starting point of the application to start its execution. When the application starts executing, the main method will be the first block of the application to be executed. The Main method contains the main logic of the application.

What is using?

Using is a keyword. Using this keyword, we can refer to .NET BCL in C# Applications i.e. including the BCL namespaces as well as we can also include user-defined namespaces that we will discuss as we progress in this course. Apart from importing the namespace, other uses of using statements are there, which we will also discuss as progress in this course. For now, it is enough.

Note: In .NET the base class libraries are divided into a collection of namespaces. Each namespace contains a set of predefined classes and sub-namespaces. The namespace contains another namespace called sub-namespaces.

Advantages of using the .NET framework from the C# point of view.

1. It provides GUI features. Earlier programming languages like C and C++ do not support GUI features but C#.NET will provide complete GUI features. All GUI features are getting from the framework.
2. We can connect with any database and perform the operations. Using ADO.NET and Entity Framework technologies, we can perform the DB operations with any Database. ADO.NET and Entity Framework are also a part of the .NET Framework.

3. The Framework also helps us to develop WEB Based applications. Using ASP.NET technology we can develop WEB Based Applications. ASP.NET itself alone cannot develop Web Applications; it requires language support. So, here we can use C# as the programming language. ASP.NET is also a part of the framework.

In the next article, I am going to discuss the [Console Class Methods and Properties](#) in detail. Here, in this article, I try to explain the **Basic Structure of the C# Program** step by step with one example. I hope you enjoy this article.